

Arquitetura de Sistemas para Indústria 4.0

Construindo um Pipeline de Dados em Python para IA (PoC Integrada)

Karina Casola

11 de maio de 2026

“Capacitar o aluno a projetar arquiteturas de integração de dados industriais utilizando Python, estruturando fluxos desde a coleta em sensores até a inferência por modelos de Inteligência Artificial, visando a otimização de processos da Indústria 4.0.

”

Integração de Sistemas Industriais

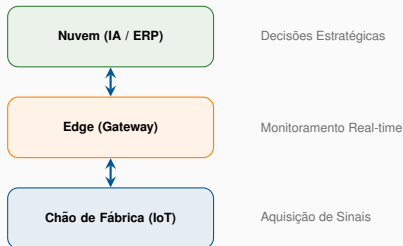
O que vamos construir hoje?

- **Compreender** a pilha tecnológica da Indústria 4.0.
- **Integrar** hardware (Arduino) com software de alto nível (Python).
- **Estruturar** um pipeline de dados capaz de alimentar modelos de IA.
- **Visualizar** dados em tempo real utilizando Brython no navegador.

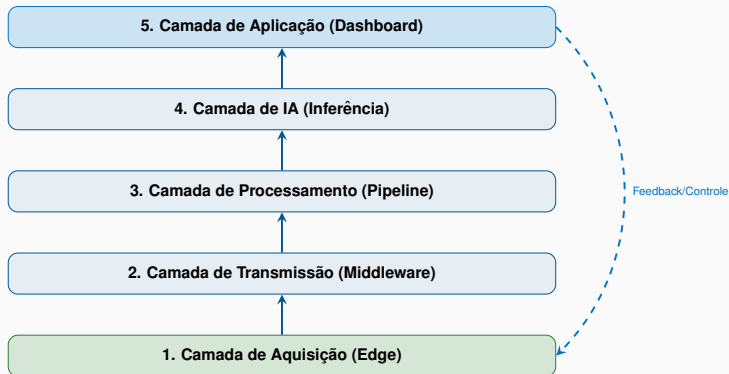
Visão Geral e Arquitetura de Referência

Hierarquia de Dados na Indústria 4.0

- **Chão de Fábrica:** Geração de dados brutos (IoT/CLP).
- **Edge Computing:** Pré-processamento e latência zero.
- **Cloud/IA:** Modelagem preditiva e Big Data.



A Pilha de 5 Camadas (Arquitetura PoC)

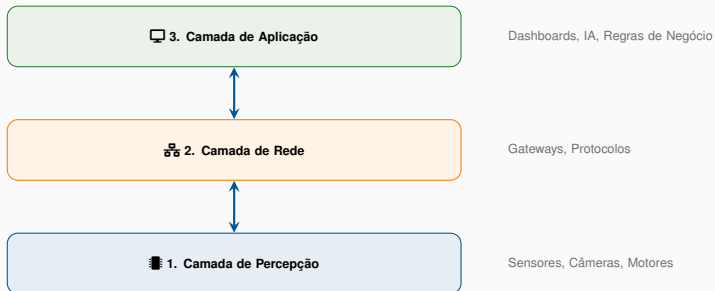


O dado flui do sensor até a decisão da IA, retornando como comando.

Camada 1 e 2: O Nascimento do Dado (IoT & Edge)

Camadas da IoT: A Estrutura Básica

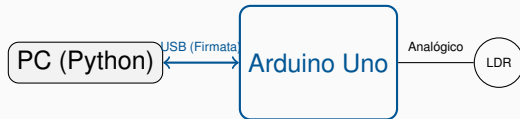
- **Camada de Percepção:** O "sentido" do sistema. Captura de dados físicos.
- **Camada de Rede:** O "sistema nervoso". Transporte seguro de dados.
- **Camada de Aplicação:** O "cérebro". Processamento e entrega de valor.



Edge Layer: Arduino + LDR (Nossa PoC)

Aplicações Práticas:

- **Sensor:** LDR (Resistor Dependente de Luz) conectado à porta A0.
- **PyFirmata2:** Protocolo que permite o Python controlar o Arduino via Serial.



Listing 1: Coleta de Luminosidade com pyFirmata2

```
from pyfirmata2 import Arduino
board = Arduino(Arduino.AUTODETECT)
sampling_rate = 100 # Hz

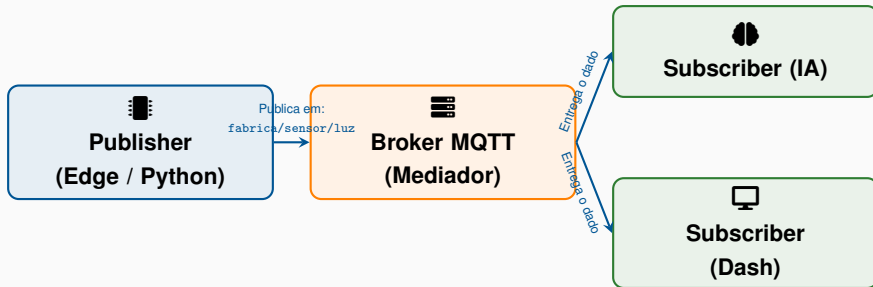
def callback(value):
    print(f"Luminosidade: {value}") # Dado estruturado (0.0 a 1.0)

board.analog[0].register_callback(callback)
board.analog[0].enable_reporting()
```

O que é o MQTT e qual seu papel na IoT?

- **MQTT** (*Message Queuing Telemetry Transport*): Protocolo de mensagens extremamente leve, ideal para sensores industriais e redes instáveis.
- **Modelo Pub/Sub**: Diferente do modelo HTTP (Requisição/Resposta), o MQTT utiliza um mediador (**Broker**). Dispositivos não conversam entre si diretamente, mas através de tópicos.
- **Eficiência**: Consome pouca bateria e pouca banda de rede, permitindo que milhares de sensores enviem dados simultaneamente para um único ponto.
- **Papel na PoC**: O Python atua como um **Publisher**, enviando o dado limpo do LDR para o Broker, que por sua vez distribui esse dado para a IA e o Dashboard.

Arquitetura de Mensagens MQTT



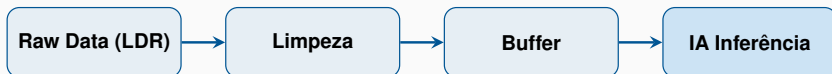
* O Broker garante que a mensagem chegue a todos os interessados sem que o sensor precise conhecê-los.

Camada 3 e 4: Inteligência e Fluxo de Dados

O Ciclo de Vida do Dado & Data Pipeline

O Pipeline é o "coração" industrial que transforma ruído em informação.

1. **Ingestão:** Coleta contínua (Streaming) via Serial ou MQTT.
2. **Pré-processamento:** *Smoothing* (Média móvel) e Normalização [0, 1].
3. **Armazenamento:** SQLite para logs históricos ou buffers em memória (Deque).



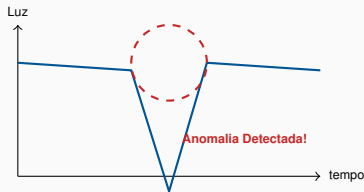
IA Layer: Detecção de Anomalias

Como aplicar IA no sensor de luz?

- **Contexto:** Queda brusca de luminosidade pode indicar falha na lâmpada ou obstrução.
- **Modelo:** Isolation Forests para detectar anomalias ou Regressão Linear.

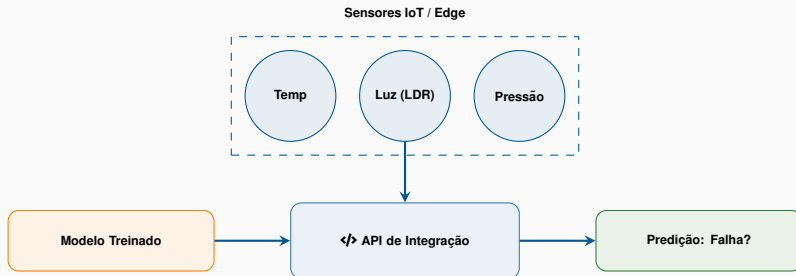
Inferência Online

O modelo recebe os últimos 10 segundos de dados do pipeline e responde: *"A operação está normal?"*



O Papel do Python

Python atua como a "cola" do sistema, integrando o protocolo industrial com o algoritmo matemático de forma escalável.



Camada 5: Controle, Feedback e Conclusão

A última camada fecha o ciclo da Indústria 4.0.

- **Visualização:** Gráficos em tempo real (Plotly ou Chart.js consumindo dados via MQTT).
- **Alertas:** Notificação visual se a luz baixar de um limiar crítico.
- **Atuadores:** Se a IA detectar luz baixa, o Python envia um comando via pyFirmata2 para ligar um LED de emergência no Arduino.

Fluxo de Feedback:

IA detecta Anomalia → Python envia sinal → Arduino liga Atuador

Resumo do Fluxo de Dados (Arquitetura PoC)

